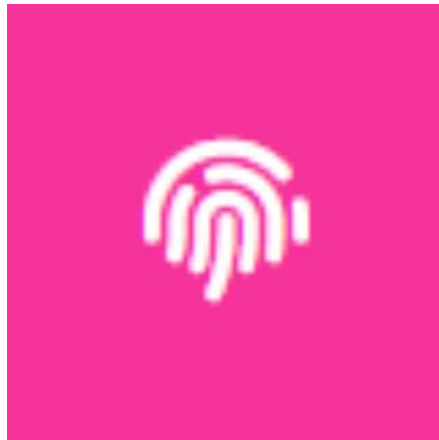




Projet réalisé dans le cadre du passage du Titre Professionnel
Développeur Web et Web Mobile (DWWM)



Projet HeartMania



Application de rencontres

Présenté par Mohamed-Amine NAMASSE
Ecole La Plateforme_ Promo Paris 2025-2026

Sommaire

Introduction.....	4
Compétences couvertes par ce projet	5
Cahier des charges :	7
1. Présentation du projet	7
2. Objectifs et positionnement	7
3. Spécifications fonctionnelles (L'expérience utilisateur).....	7
3.1. Système d'authentification	7
3.2. Espace personnel	7
3.3. Découverte et interactions	7
3.4. Monétisation	8
3.5. Sécurité et gestion de la communauté.....	8
3.6. Internationalisation.....	8
4. Spécifications techniques	8
4.1. Architecture frontend (le client).....	8
4.2. Architecture backend (le serveur)	8
4.3. Base de données et services cloud.....	9
5. Contraintes et Conformité	9
6. Fonctionnalités	9
La réalisation du projet : HeartMania.....	11
1. Frontend	11
1.1 Main.jsx	11
1.2 Index.html	11
1.3 App.jsx.....	14
1.4 Structure de l'application	15
1.5 Authentification.....	19
1.6 Achats (Système stripe)	26
1.7 Gestion d'inactivité ,suppression des comptes et gestion des sessions	31
1.8. Suppression des comptes fantômes et ban	31
2. La conception du backend	32
3. Performance & accessibilité.....	38

Conclusion	40
------------------	----

Introduction

Ayant toujours eu envie de concevoir et déployer ma propre application mobile, j'ai décidé de concrétiser cette ambition en créant l'application HeartMania. Ce projet est né d'une volonté de proposer une alternative moderne et sécurisée dans le secteur très compétitif des plateformes de mise en relation. HeartMania est une application mobile de rencontre spécifiquement pensée pour connecter un public adulte de manière fiable, fluide et instantanée.

La réalisation de ce travail n'aura pu être possible sans avoir eu les acquis nécessaires lors de mon apprentissage durant la formation CDPI Développeur Web et Web Mobile (DWWM) à l'Ecole LaPlateforme_ au Campus de Paris.

Compétences couvertes par ce projet

I. Développer la partie front-end d'une application web ou web mobile sécurisée

❖ Installer et configurer son environnement de travail en fonction du projet web ou web mobile

Pour lancer le développement de Heartmania, j'ai mis en place un espace de travail moderne et structuré. J'ai configuré le projet avec l'outil de build Vite pour coder de manière fluide, et j'ai géré l'installation de toutes les extensions nécessaires (pour la navigation, les icônes, le style, le paiement stripe et les outils Firebase, etc..) via le gestionnaire de paquets de Node.js. Pour sécuriser l'application, j'ai isolé les clés de configuration privées de Firebase dans des fichiers masqués (.env) afin qu'elles ne soient jamais visibles publiquement. Enfin, j'ai installé un système de traduction (i18n) pour traduire l'interface de l'application à la volée.

❖ Réaliser des interfaces utilisateur statiques web ou web mobile

J'ai conçu l'habillage visuel et la structure de Heartmania en adoptant une approche Mobile-First, garantissant que l'application soit parfaitement utilisable sur smartphone tout en restant élégante sur ordinateur. En utilisant le framework Tailwind CSS, j'ai intégré toute la mise en page des écrans clés comme le profil utilisateur ou la page d'accueil. Cela m'a permis de créer des éléments visuels soignés et modernes : des fenêtres modales pour les confirmations, des animations fluides sur les menus, et un panneau latéral pour l'affichage des notifications.

❖ Développer la partie dynamique des interfaces utilisateur web ou web mobile

Pour rendre Heartmania interactive, j'ai programmé toute la logique de fonctionnement des écrans. J'ai notamment développé le système de commentaires et de réponses pour qu'il réagisse correctement lorsqu'un utilisateur interagit avec un post. J'ai veillé à ce que l'ajout de texte, les focus de curseurs et les touches "Entrée" s'activent de manière isolée sur chaque publication. L'application envoie et reçoit ainsi les données de manière invisible et dynamique, offrant à l'utilisateur des messages clairs de confirmation (succès ou erreur) après chaque action.

II. Développer la partie back-end d'une application web ou web mobile sécurisée

❖ Mettre en place une base de données

Pour enregistrer toutes les données de Heartmania, j'ai organisé l'architecture d'une base de données cloud (NoSQL avec Firestore). Pour assurer d'excellentes performances de chargement, j'ai modélisé la structure pour que les commentaires et leurs réponses soient directement rattachés à leur publication d'origine. Côté sécurité et confidentialité, j'ai compartimenté l'historique des notifications pour qu'il soit stocké dans un espace propre à chaque utilisateur, garantissant que personne ne puisse lire les notifications d'un autre profil.

❖ Développer des composants d'accès aux données SQL et NoSQL

J'ai écrit le code chargé de faire la passerelle entre l'interface utilisateur et la base de données. J'ai mis en place des requêtes spécifiques pour trier automatiquement les publications par ordre chronologique ou filtrer les notifications afin de n'afficher que les alertes non lues. Pour sécuriser les interactions et éviter les conflits d'écriture, j'ai utilisé des fonctions de mise à jour

sécurisées qui permettent d'ajouter un commentaire, d'aimer un post ou de bloquer un utilisateur sans risquer d'effacer par erreur d'autres données.

❖ **Développer des composants métier côté serveur**

Bien que la base de données soit hébergée dans le cloud, j'ai configuré les règles et la logique métier indispensables pour protéger l'application Heartmania. J'ai intégré le système de gestion des comptes (connexion avec compte Google, avec numéro de téléphone ou en mode invité anonyme). J'ai également codé la logique des notifications : l'application détecte si l'action est un commentaire global ou une réponse ciblée, et transmet l'alerte à la bonne personne. Enfin, l'enregistrement du moment exact de chaque action s'appuie sur l'heure du serveur pour empêcher toute triche ou décalage horaire entre les téléphones.

Cahier des charges :

1. Présentation du projet

- **Nom de l'application :** HeartMania
- **Secteur d'activité :** Plateforme de mise en relation / Rencontres
- **Public cible :** Public adulte (18 ans et plus)
- **Objectif principal :** Connecter des utilisateurs majeurs de manière fiable, fluide, instantanée et hautement sécurisée.

2. Objectifs et positionnement

HeartMania vise à offrir une alternative moderne aux applications de rencontre traditionnelles. Le projet privilégie la légèreté technique, le confort visuel, la gamification de la mise en relation, le bilinguisme (Français/Anglais), ainsi qu'un respect strict des directives de sécurité imposées par Google Play Store.

3. Spécifications fonctionnelles (L'expérience utilisateur)

3.1. Système d'authentification

Pour maximiser l'accessibilité tout en garantissant la sécurité, l'application propose trois modes de connexion distincts :

- **Connexion Google:** Authentification rapide et sécurisée en un clic via le compte Google de l'utilisateur.
- **Connexion par téléphone (SMS) :** Authentification par numéro de mobile avec envoi d'un code de vérification unique, renforçant la sécurité et la certification des profils.
- **Connexion en mode Invité:** Mode qui permet d'accéder à l'interface et d'explorer l'application sans inscription immédiate, afin de réduire la friction à l'entrée.

3.2. Espace personnel

- **Page profil :** Espace vitrine de l'utilisateur où il peut consulter ses informations, mais aussi rédiger et publier des posts visibles par la communauté.
- **Visite de profils tiers :** Possibilité de cliquer sur un utilisateur pour consulter son profil public, découvrir sa description et accéder directement au fil de ses publications personnelles.
- **Système de commentaires :** Module d'interaction complet permettant de commenter et de liker les posts de n'importe quel utilisateur (sur son propre profil ou sur celui d'un autre), favorisant des discussions ouvertes avant ou après un Match.

3.3. Découverte et interactions

- **Système de swipe :** Défilement fluide de profils permettant d'aimer (Like) ou de passer (Dislike) les utilisateurs suggérés.
- **Système de superlikes :** Possibilité d'envoyer une mention spéciale pour se démarquer et notifier prioritairement un profil à fort intérêt.

- **Messagerie et temps réel** : Échange de messages instantanés en cas de compatibilité (Match), avec mise à jour en temps réel des statuts de lecture des messages reçus.

3.4. Monétisation

- **La boutique** : Espace dédié permettant l'achat des packs de gems afin d'optimiser les chances de rencontre.

3.5. Sécurité et gestion de la communauté

- **Système de blocage de profil** : Possibilité pour chaque utilisateur de restreindre instantanément l'accès à son espace privé et couper tout contact avec un profil indésirable.
- **Système de signalement** : Possibilité de notifier l'équipe de modération en cas de comportement abusif.
- **Protection anti-spam intégrée** : Limitation stricte des signalements par jour et par profil pour éviter le harcèlement ciblé et réguler les requêtes.

3.6. Internationalisation

Traduction dynamique et instantanée de l'interface en français et en anglais via i18n, ce qui permet de s'adapter aux préférences de l'utilisateur et de toucher un large public.

4. Spécifications techniques

J'ai utilisé l'IDE Visual Studio Code pour faire le développement de ce projet. Ci-dessous on retrouve l'architecture du front-end et du back-end.

4.1. Architecture frontend (le client)

- **Framework / Bibliothèque** : React
- **Outil de build et de développement** : Vite
- **Hébergement** : Vercel (consultable via le lien : <https://heartmania.vercel.app>)
- **Format de déploiement web** : PWA (Progressive Web App) pour assurer la légèreté de l'interface, une gestion intelligente de la mémoire et l'accès hors-ligne partiel.
- **Format de déploiement mobile** : TWA (Trusted Web Activity) pour encapsuler la PWA et permettre une publication native sur le Google Play Store.

4.2. Architecture backend (le serveur)

- **Environnement** : Node.js avec le framework Express.
- **Hébergement** : Serveur distant Alwaysdata.
- **Rôle** : Centralisation de la logique métier sensible (validation des achats de la boutique, distribution des Superlikes, authentification, modération et gestion des statuts de lecture mark-read). Tout est soumis au contrôle strict du serveur.

4.3. Base de données et services cloud

- **Technologie** : Firebase / Firestore.
- **Gestion des utilisateurs** : Authentification sécurisée gérée via *Firebase Authentication* pour l'intégration native des flux Gmail, Téléphone (SMS OTP) et Comptes Anonymes (Guest).
- **Sécurité des requêtes** : Génération et vérification systématique de jetons sécurisés (getIdToken) par le backend Alwaysdata pour chaque profil authentifié.
- **Optimisation des performances** : Mise en place d'index composites Firestore pour exécuter les requêtes complexes de sécurité et l'affichage des posts/swipes sans ralentissement.

5. Contraintes et Conformité

- **Sécurité et Transactions** : Chiffrement et vérification systématique des droits d'accès à l'API via les jetons Firebase, notamment pour la gestion de la boutique et la transition d'un compte "Guest" vers un compte permanent.
- **Conformité Store** : Respect total des exigences du Google Play Store concernant la protection des utilisateurs au sein des applications de rencontre pour adultes et la gestion transparente des options payantes.

6. Fonctionnalités

Lors de la création de cette application j'ai travaillé avec la méthode agile en suivant des sprints réguliers.

MVP

Ci-dessous la liste des fonctionnalités minimales qui donne un produit utilisable

ID	En tant que...	Je veux...	Afin de...
US-01	Utilisateur	Me connecter rapidement avec mon compte Google (Gmail)	Accéder instantanément à l'application sans remplir de long formulaire.
US-02	Utilisateur	M'authentifier en saisissant mon numéro de téléphone	Recevoir un code de validation unique par SMS pour certifier mon accès.
US-03	Visiteur	Explorer l'application en mode "Invité" (Guest)	Découvrir l'interface et les fonctionnalités avant de décider de m'inscrire.
US-04	Utilisateur	Pouvoir faire défiler (swiper) les profils suggérés	Indiquer rapidement si une personne m'intéresse (droite) ou non (gauche).
US-06	Utilisateur	Envoyer un "Superlike" à un profil spécifique	Me démarquer et lui notifier prioritairement mon grand intérêt.

US-07	Utilisateur	Rédiger une biographie sur ma page profil	Faire une petite présentation aux gens qui consultent mon profil
US-10	Utilisateur	Voir le profil d'un autre utilisateur	Pouvoir voir les détails concernant un profil
US-12	Utilisateur	Envoyer un message à un profil via le chat	Echanger avec une personne que j'ai matchée
US-13	Utilisateur	Voir le statut d'un utilisateur	Savoir si un utilisateur est en ligne ou hors ligne
US-14	Utilisateur	Pouvoir bloquer un profil	Empêcher la personne bloquée de m'envoyer des messages
US-15	Utilisateur	Voir la liste des profils bloqués sur ma page profil	Consulter la liste des profils que j'ai bloqué
US-16	Utilisateur	Signaler un profil	Notifier l'équipe de modération pour qu'elle examine la situation.
US-17	Utilisateur	Me rendre dans la boutique	Acheter des packs de gems
US-19	Utilisateur	Pouvoir supprimer mon compte	Supprimer toutes les données de mon compte de façon définitive
US-21	Admin	Pouvoir bannir un compte	Bannir définitivement un compte

Fonctionnalité qui enrichit l'app

ID	En tant que...	Je veux...	Afin de...
US-05	Utilisateur	Pouvoir filtrer les personnes dans le swipe	Filtrer les profils par âge, pays, genre et dernière activité
US-08	Utilisateur	Rédiger et publier des posts sur ma page profil	Partager mon humeur, mes actualités et attirer l'attention des autres membres.
US-09	Utilisateur	Voir qui a visité et/ou envoyé un Superlike à mon profil	Consulter la liste des visiteurs et des profils qui m'ont envoyé un superlike directement via ma page profil
US-11	Utilisateur	Voir les posts d'un membre et pouvoir liker et/ou laisser un commentaire	Engager la conversation sur un posts ou un commentaire
US-18	Utilisateur	Cacher une discussion	Afficher que les discussions qui m'intéresse
US-20	Utilisateur	Pouvoir choisir la langue de l'application	Utiliser l'application dans ma langue préférée
US-22	Utilisateur	Pouvoir modifier la photo de profil	Modifier la photo de profil

La réalisation du projet : HeartMania

1. Frontend

1.1 Main.jsx

Le fichier "main.jsx" est le point d'entrée principal d'une application React. Il initialise le rendu de l'interface en ciblant l'élément HTML d'identifiant root et enveloppe le composant racine (App) dans un BrowserRouter, ce qui permet la gestion du routage côté client pour l'application. L'utilisation de StrictMode active des contrôles supplémentaires pour identifier les problèmes potentiels durant le développement, tandis que les importations de fichiers (CSS et i18n) assurent le chargement des styles globaux et de la configuration de l'internationalisation nécessaires au fonctionnement de l'application.

```
1  import { StrictMode } from "react";
2  import { createRoot } from "react-dom/client";
3  import { BrowserRouter } from "react-router-dom";
4  import "./i18n";
5  import "./index.css";
6  import "./App.css";
7
8  import App from "./App.jsx";
9
10 createRoot(document.getElementById("root")).render(
11   <StrictMode>
12     <BrowserRouter>
13       <App />
14     </BrowserRouter>
15   </StrictMode>,
16 );
```

1.2 Index.html

Comme j'ai utilisé Google Search Console pour indexer mon site sur le moteur Google, j'ai ajouté les balises meta de google pour l'indexation et j'ai bien développé la balise description.

J'ai également inclus un fichier sitemap.xml et robots.txt pour avoir un bon référencement SEO.

J'ai transformé mon app en PWA en utilisant la dépendance vite-plugin-pwa et en ajoutant dans le <head> la balise <link rel="manifest" href="/manifest.webmanifest">

L'élément avec l'ID "root" dans le fichier "index.html" sert de point d'ancrage pour le rendu de l'application React. C'est un élément HTML où le contenu généré par React sera inséré.

L'utilisation des balises `<link rel="preconnect" ... />` servent à optimiser les performances de chargement du site en préparant la connexion avec des serveurs tiers avant même que l'application ait réellement besoin de télécharger des fichiers depuis ces serveurs.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" href="/pwa-192x192.png" />
    <link rel="manifest" href="/manifest.webmanifest" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0, viewport-fit=cover"
    />
    <meta
      name="google-site-verification"
      content="4zEBVWlIvMahzjNV3dMpQF0f8BmeOTAYljmy9kt5kS0"
    />
    <meta
      name="description"
      content="La Web App pour se faire des amis . Découvrez des profils, discutez et trouvez votre âme
    />
    <title>HeartMania</title>

    <link rel="preconnect" href="https://firestore.googleapis.com" />
    <link rel="preconnect" href="https://identitytoolkit.googleapis.com" />
    <link rel="preconnect" href="https://flagcdn.com" />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Pour la mise en place de la PWA, j'ai importé le plugin VitePWA dans le fichier config vite

```

1  import { defineConfig } from "vite";
2  import react from "@vitejs/plugin-react";
3  import tailwindcss from "@tailwindcss/vite";
4  import { VitePWA } from "vite-plugin-pwa";
5
6  export default defineConfig({
7    plugins: [
8      react(),
9      tailwindcss(),
10     VitePWA({
11       registerType: "prompt",
12       filename: "sw.js", // Optionnel : force le nom du service worker
13       manifest: {
14         name: "HeartMania",
15         short_name: "HeartMania",
16         icons: [
17           {
18             src: "pwa-512x512.png",
19             sizes: "512x512",
20             type: "image/png",
21           },
22         ],
23       },
24       devOptions: {
25         enabled: true, // Permet de tester en local
26       },
27     }),
28   ],
29

```

Pout avoir un affichage le plus natif possible sur mobile comme il s'agit d'une PWA j'ai inclus un fichier assetlinks.json. Ce fichier comporte le nom et l'empreinte SHA-256 du certificat de signature du package de l'application HeartMania. Par conséquent, une fois Chrome valide ce fichier; il lance l'application sans aucune interface de navigateur (mode "Chromeless").

```

() assetlinks.json •
frontend > public > .well-known > {} assetlinks.json > ...
1  [
2    {
3      "relation": ["delegate_permission/common.handle_all_urls"],
4      "target": {
5        "namespace": "android_app",
6        "package_name": "fr.heartmania.twa",
7        "sha256_cert_fingerprints":
8          ["62:CD:5F:C1:43:E2:7C:B7:DF:18:7B:7F:99:88:D6:11:38:96:25:C4:50:FE:F4:D9:9A:43:3F:65:4E:B4:33:82"]
9      }
10   }
11 ]

```

1.3 App.jsx

Le fichier App.jsx agit comme le cœur transactionnel et logique de l'application. C'est celui qui orchestre la navigation, l'authentification et la gestion des données Firebase.

Voici les rôles principaux de ce fichier :

- **Gestion du Routage (react-router-dom)** : Il définit la structure de navigation de l'application via Routes et Route. Il gère non seulement les sections principales (Discover, Shop, Cart, Profile), mais aussi les pages de retour après paiement (Success, Cancel).
- **Hub d'Authentification (Firebase Auth)** : Il centralise la gestion des utilisateurs. Il surveille l'état de connexion (onAuthStateChanged) et gère les différentes méthodes d'accès, qu'il s'agisse de Google (signInWithPopup), d'un numéro de téléphone ou d'une connexion anonyme.
- **Interactions avec Firestore** : Il importe une vaste gamme d'outils Firebase (lecture, écriture, mise à jour en temps réel via onSnapshot) nécessaires pour synchroniser les données de l'application, comme les profils, les paniers ou les messages.
- **Interface et Expérience Utilisateur** : Il importe les icônes (react-icons/fa6) pour enrichir visuellement l'interface et utilise i18next pour garantir que l'application est multilingue.
- **Coordination des composants** : Il importe et réunit les composants métier (comme NavButton pour la navigation, UpdatePrompt pour les alertes, ou UserProfile) pour les rendre accessibles dans le flux de l'application.

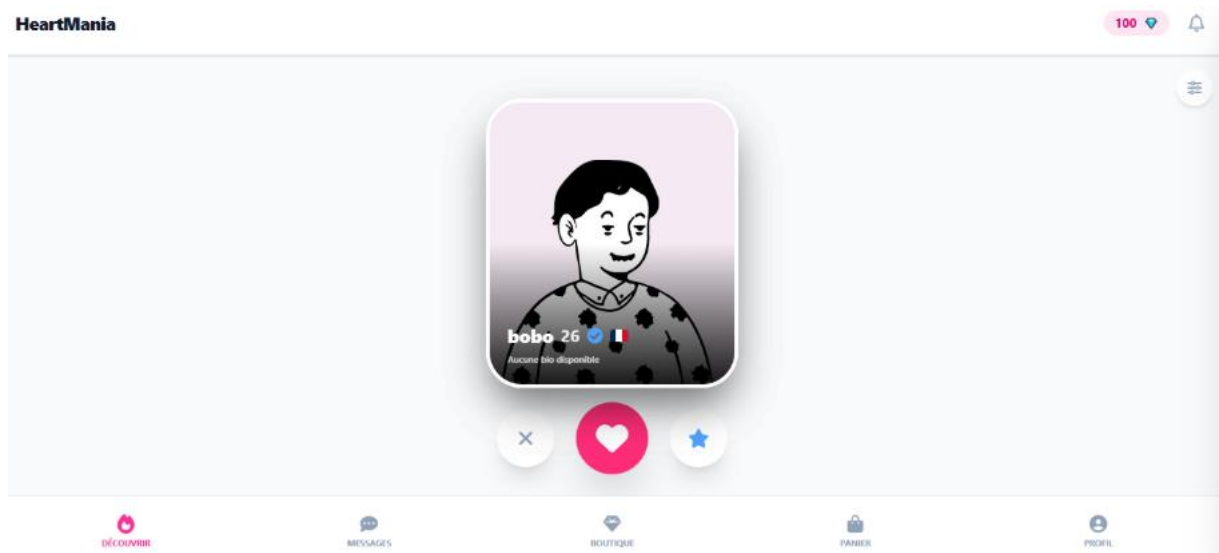
```
1 import { useState, useEffect, useRef } from "react";
2 import { getUniqueUserImage } from '../utils/image';
3 import NavButton from "../components/NavButton";
4 import Discover from "../components/Discover";
5 import Shop from "../components/Shop";
6 import Messages from "../components/Messages";
7 import Cart from "../components/Cart";
8 import Profile from "../components/Profile";
9 import UserProfile from "../components/UserProfile";
10 import UpdatePrompt from "../components/UpdatePrompt";
11 import { auth, googleProvider, db } from "../firebase-config";
12 import { Routes, Route, useNavigate, useLocation } from "react-router-dom";
13 import Success from "../pages/Success";
14 import Cancel from "../pages/Cancel";
```

```
import {
  FaSpinner, FaFingerprint, FaGoogle, FaPhone, FaAt, FaCalendarDays, FaXmark, FaCircleCheck,
  FaRegBell, FaCheck, FaFire, FaCommentDots, FaGem, FaBagShopping, FaCircleUser,
} from "react-icons/fa6";
import {
  signInWithPopup,
  onAuthStateChanged,
  signOut,
  signInWithPhoneNumber,
  RecaptchaVerifier,
  signInAnonymously,
} from "firebase/auth";
import {
  query, where, collection, getDocs, doc, getDoc, onSnapshot, setDoc, addDoc,
  updateDoc, arrayRemove, writeBatch,
} from "firebase/firestore";
import { serverTimestamp } from "firebase/firestore";
import { useTranslation } from "react-i18next";
```

1.4 Structure de l'application

Pour répondre aux objectifs du projet HeartMania, j'ai conçu et développé une application web mobile intuitive, organisée autour d'une navigation fluide par onglets. Afin d'offrir une expérience utilisateur optimale et calquée sur les standards des applications mobiles actuelles, j'ai structuré l'architecture de l'interface en cinq sections principales :

- **L'onglet Découvrir** : C'est la page d'accueil de l'application. J'ai développé cet écran autour de la fonctionnalité de "swipe" (défilement/balayage). Cet onglet permet aux utilisateurs de parcourir de manière dynamique les différents profils.

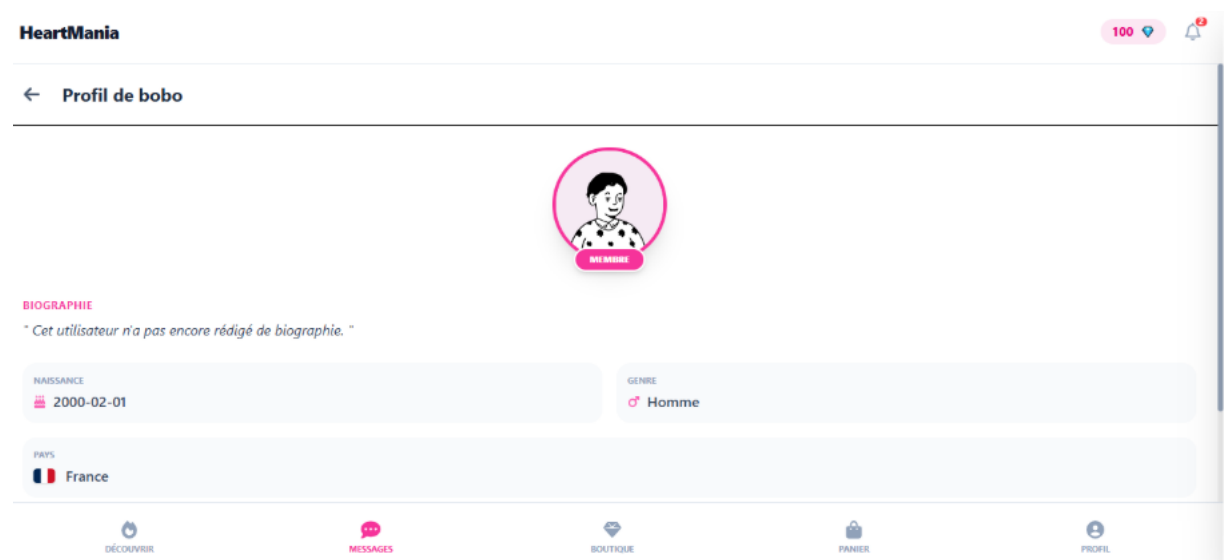


Pour avoir un défilement natif (version mobile), en plus du défilement classique qui consiste à juste cliquer sur les boutons, j'ai installé la dépendance react-tinder-card.

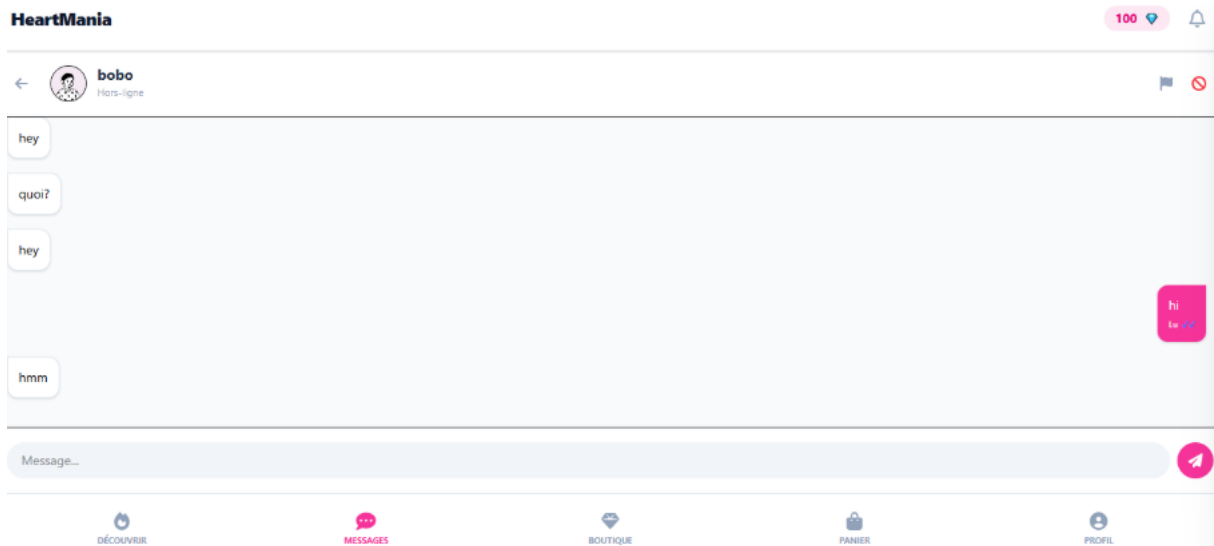
- **L'onglet Messages (La messagerie) :** Cet espace est dédié à la communication. J'ai réalisé cette interface pour regrouper l'ensemble des discussions privées des utilisateurs. Cet onglet permet de suivre les conversations en cours, de centraliser les échanges et de maintenir le lien social entre les membres de la communauté.



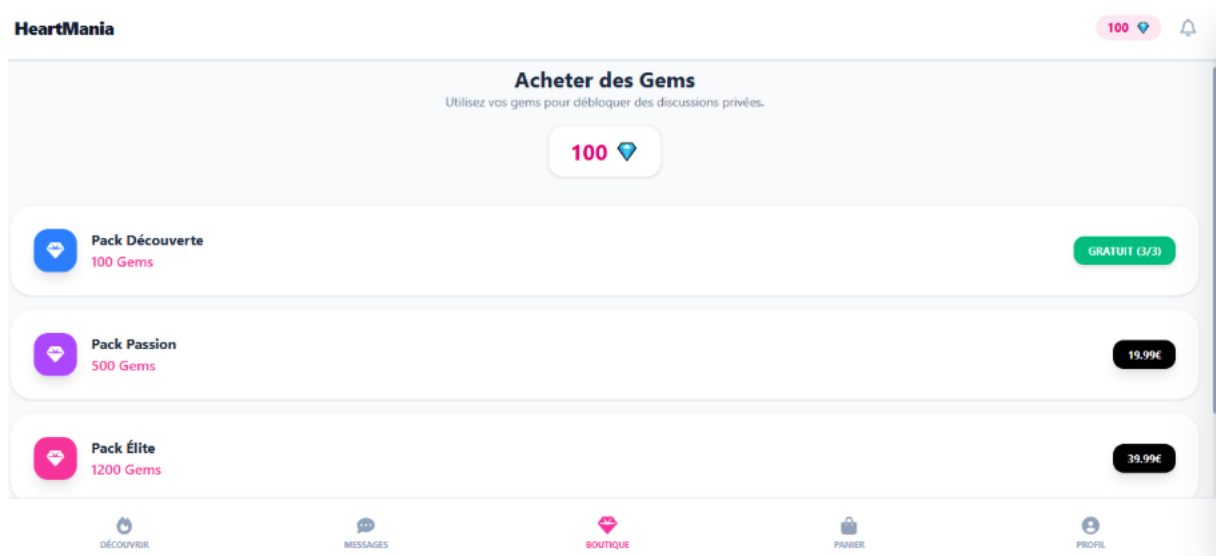
L'utilisateur peut consulter bloquer/signaler un profil, cacher une discussion et consulter le profil d'un autre utilisateur en cliquant sur son image



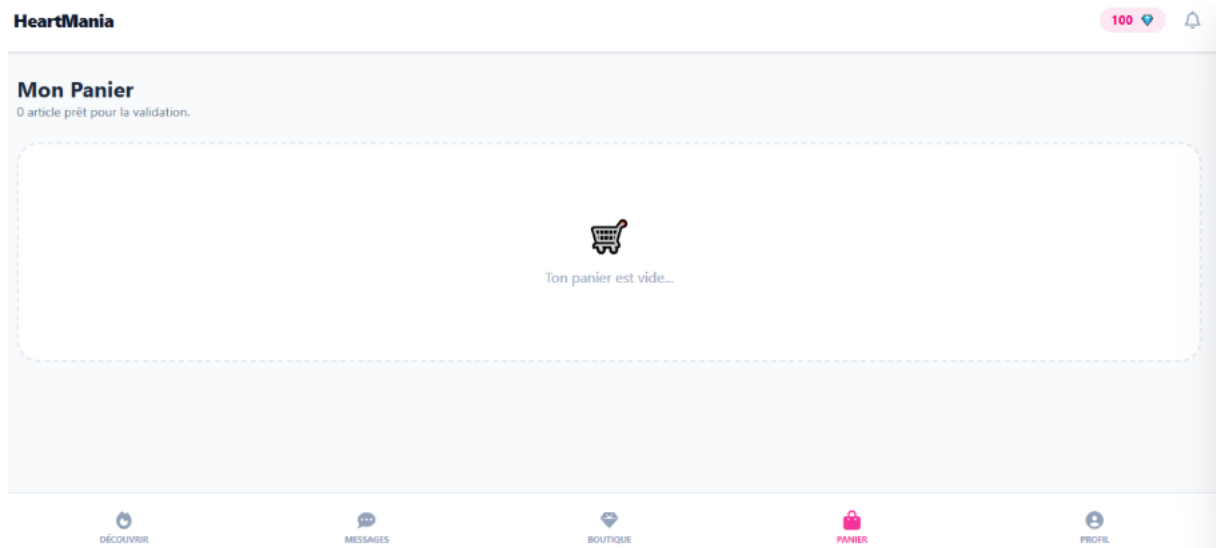
Ci-dessous l'aperçu quand quelqu'un lance une discussion



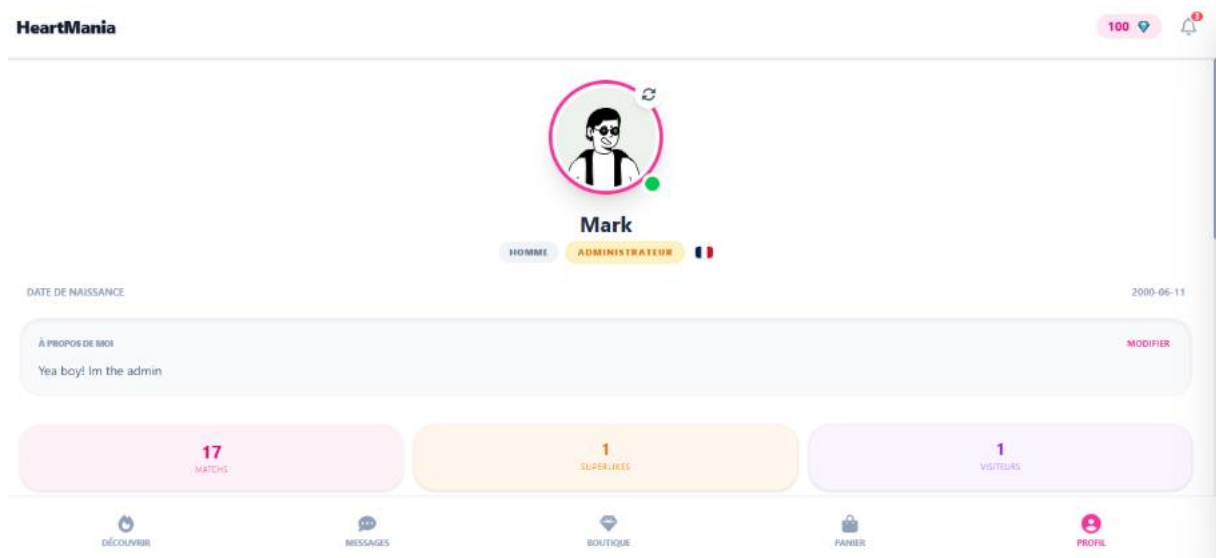
- **L'onglet Boutique:** Cet onglet fait office de vitrine pour l'application. J'ai conçu cet écran pour présenter les différents packs disponibles à l'achat. L'accent a été mis sur la clarté de l'affichage des produits pour permettre une navigation commerciale agréable et engageante.



- **L'onglet Panier :** Directement lié à la boutique, cet écran centralise les choix de l'utilisateur avant la validation de ses commandes. J'ai structuré cet onglet pour qu'il affiche un récapitulatif clair des articles sélectionnés, des quantités et des prix, garantissant ainsi une expérience d'achat sécurisante et transparente.

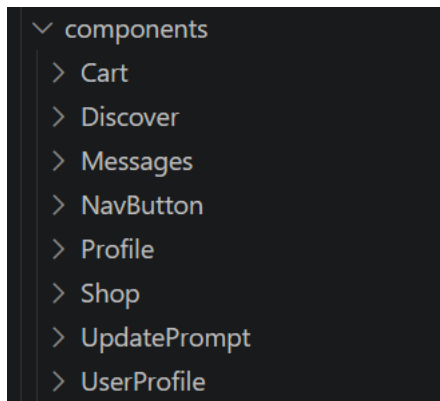


- **L'onglet Profil (La Gestion utilisateur) :** C'est l'espace personnel de l'utilisateur, que j'ai équipé de fonctionnalités avancées. Cet onglet permet non seulement de personnaliser ses informations (photos, biographie, pays), mais il intègre également le fil de ses propres publications, son historique de notifications en temps réel (commentaires et réponses reçus), ainsi que des options de sécurité essentielles comme la gestion des utilisateurs bloqués ou la suppression de compte.



Pour les icones j'ai utilisé la dépendance react-icons.

Coté code comme je suis parti sur une SPA (single page application) les onglets ont été développés sous la forme de composants directement importés dans App.jsx.



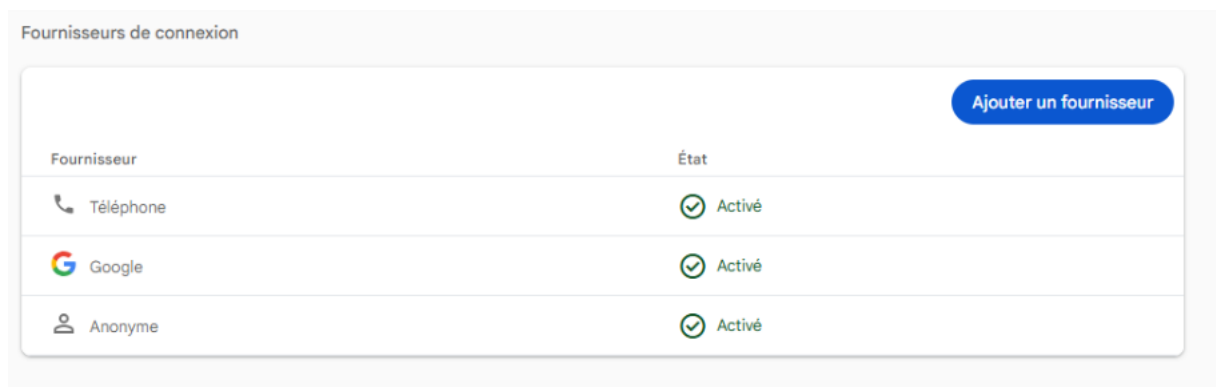
Et j'ai deux pages en plus qui sont gérés par react-router-dom, qui sont les pages de succès et d'échec après une transaction stripe.



1.5 Authentification

Après avoir conçu les différentes pages. J'ai pensé à implémenter le système de connexion

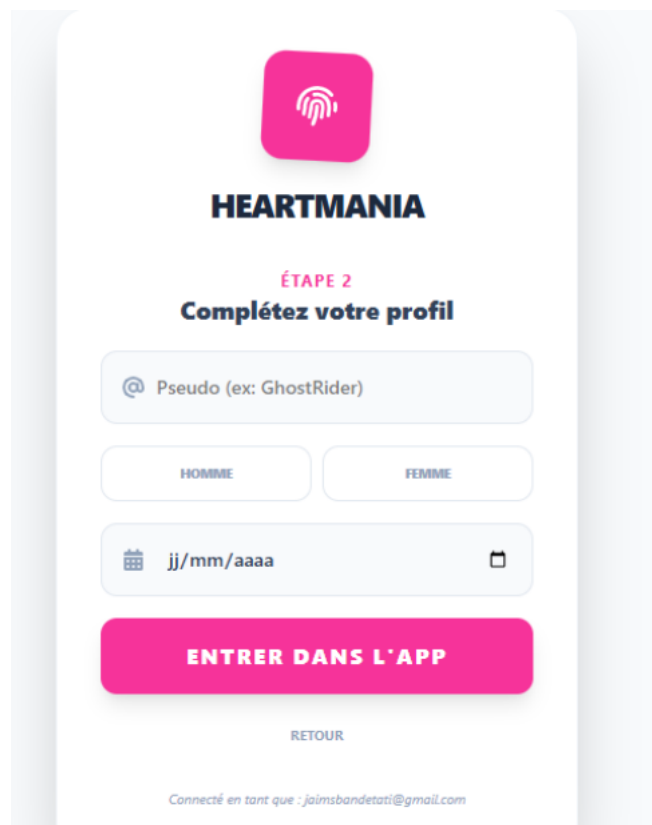
J'ai choisi trois modes de connexion (connexion via compte Google, par téléphone et en mode invité)



Le cheminement de connexion/inscription est le suivant :

L'étape 1 : l'utilisateur se connecte par l'une des méthodes évoquées précédemment, puis il passe à l'étape 2.

L'étape 2 : l'utilisateur doit remplir un formulaire pour créer son profil. Il est judicieux de préciser qu'une fois le profil créé il obtient une image de profil générée de façon aléatoire via l'API DiceBear (API open-source spécialisée dans la génération d'avatars personnalisables, conçue pour les développeurs et les designers)



❖ Fonction pour la connexion des invités

```
// --- 3. AUTHENTIFICATION (Front) ---
const handleGuestLogin = async () => {
  try {
    setIsProcessing(true);
    await signInAnonymously(auth);
    showToast(t("welcome_guest"));
  } catch (error) {
    console.error("Erreur connexion anonyme:", error);
    showToast("Erreur lors de la connexion Invité ✗");
  } finally {
    setIsProcessing(false);
  }
};
```

Si le compte est un compte invité, il obtient un pseudo qui commence par Guest suivi d'un numéro généré aléatoirement.

```
if (currentUser && currentUser.isAnonymous) {
  const randomId = Math.floor(1000 + Math.random() * 9000);
  setLoginName(`Guest_${randomId}`);
}
```

❖ Fonctions de connexion compte Google

```
const handleGoogleLogin = async () => {
  try {
    const result = await signInWithPopup(auth, googleProvider);
    const userUID = result.user.uid;
    const userRef = doc(db, "users", userUID);
    const userSnap = await getDoc(userRef);

    const profile = result._tokenResponse;
    const notifRef = collection(db, "users", userUID, "notifications");
    const currentTime = new Date().toISOString();

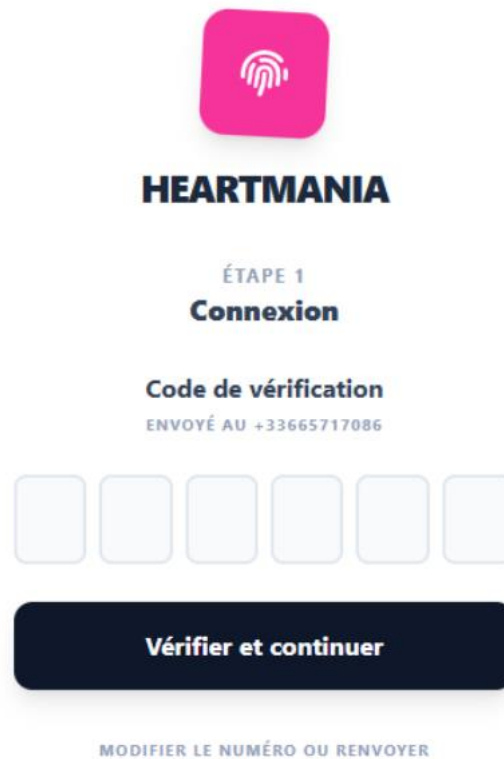
    if (userSnap.exists()) {
      // --- CAS : RETOUR (Déjà inscrit) ---
      const data = userSnap.data();
      await updateDoc(userRef, { isOnline: true });

      await addDoc(notifRef, {
        id: `welcome-back-${Date.now()}`,
        read: false,
        text: "welcome_back",
        time: currentTime,
        timestamp: new Date(),
        type: "info",
      });
    }
  }
};
```

```
    } else {
      // --- CAS :Direction Étape 2 ---
      // On stocke les infos Google de base pour aider au remplissage
      setTempAuth({
        uid: userUID,
        name: result.user.displayName,
        email: result.user.email,
        img: result.user.photoURL,
        gender: profile.gender || "",
        birthday: "", // La date est vide, il va la mettre à l'étape 2
      });
      setStep(2);
    }
  } catch (error) {
    console.error("Erreur Google:", error);
    showToast("Erreur Google ❌");
  }
};
```

❖ Fonctions de connexion par téléphone

J'ai implémenté un système de captcha avant l'envoi de SMS qui contient le code à usage unique



The image shows a mobile app login screen for 'HEARTMANIA'. At the top is a pink square icon with a white fingerprint symbol. Below it, the text 'HEARTMANIA' is displayed in bold. Underneath is 'ÉTAPE 1' followed by 'Connexion' in bold. The main heading is 'Code de vérification' with a subtitle 'ENVOYÉ AU +33665717086'. Below this are six empty input boxes for a 6-digit code. A dark blue button with the text 'Vérifier et continuer' is positioned below the boxes. At the bottom, there is a link that says 'MODIFIER LE NUMÉRO OU RENVoyer'.

```
const handleSendSMS = async () => {
  try {
    if (window.recaptchaVerifier) window.recaptchaVerifier.clear();
    window.recaptchaVerifier = new RecaptchaVerifier(
      auth,
      "recaptcha-container",
      { size: "invisible" },
    );
    const confirmation = await signInWithPhoneNumber(
      auth,
      phoneNumber,
      window.recaptchaVerifier,
    );
    setConfirmationResult(confirmation);
    setAuthMode("otp");
    showToast(t("sms_sent"));
  } catch (error) {
    showToast(t("error_invalid_number"));
  }
};
```

```

const handleVerifyOTP = async () => {
  try {
    if (confirmationResult) {
      const userCredential = await confirmationResult.confirm(otp);
      const uid = userCredential.user.uid;

      const notifCollectionRef = collection(
        db,
        "users",
        uid,
        "notifications",
      );
      await addDoc(notifCollectionRef, {
        id: `welcome-back-${Date.now()}`,
        read: false,
        text: "welcome_back",
        time: new Date().toISOString(),
        timestamp: serverTimestamp(),
        type: "info",
      });

      showToast(t("welcome_back"));
    }
  } catch (error) {
    showToast(t("error_invalid_code"));
  }
};

```

❖ Fonctions d'inscription

Après avoir terminé l'étape 1, à l'étape 2 l'utilisateur doit remplir le formulaire (nom d'utilisateur, sexe, date de naissance)

J'ai mis en place un regex (expression régulière) pour la validation du champ nom d'utilisateur.

Pour les utilisateurs normaux on autorise les lettres, les espaces et les underscores.

Pour les invités on autorise les lettres, les espaces et les underscores et les chiffres.

Le fait que les chiffres ne sont pas autorisés pour les utilisateurs normaux a été fait dans le but d'améliorer l'expérience utilisateur.

```

const handleFinalizeLogin = async () => {
  const isGuest = auth.currentUser.isAnonymous;
  // 1. VALIDATIONS DE STRUCTURE
  const cleanName = loginName.trim();

  const pseudoRegex = isGuest
    ? /^[a-zA-Z0-9_]{3,15}$/ // Pour les invités (lettres, chiffres, espaces, underscores)
    : /^[a-zA-Z_]{3,15}$/; // Pour les officiels (pas de chiffres)

  if (cleanName.length < 3) return alert(t("error_pseudo_short"));
  if (cleanName.length > 15) {
    return alert(t("error_pseudo_long"));
  }
  if (!pseudoRegex.test(cleanName)) {
    return alert(t("regex"));
  }

  if (!gender) return alert(t("error_no_gender"));
  if (!birthDate) return alert(t("error_no_birthdate"));

```

```

// --- 2. CALCUL ET VÉRIFICATION DE L'ÂGE ---
const birth = new Date(birthDate);
const today = new Date();
let age = today.getFullYear() - birth.getFullYear();
const monthDiff = today.getMonth() - birth.getMonth();
if (
  monthDiff < 0 ||
  (monthDiff === 0 && today.getDate() < birth.getDate())
) {
  age--;
}

if (age < 18) {
  alert(t("error_age_restriction", { age }));
  return;
}
try {
  if (!auth.currentUser) return showToast(t("error_session_expired"));

  const token = await auth.currentUser.getIdToken();

  // APPEL SERVEUR
  const response = await fetch(`${API_URL}/api/auth/register`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",

```



```

    Authorization: `Bearer ${token}`,
  },
  body: JSON.stringify({
    name: cleanName,
    gender,
    birthDate,
    age,
    isAnonymous: auth.currentUser.isAnonymous,
  }),
});

const serverData = await response.json();

if (response.ok) {
  // --- AJOUT DE LA NOTIFICATION DE BIENVENUE DANS FIRESTORE ---
  const notifCollectionRef = collection(
    db,
    "users",
    auth.currentUser.uid,
    "notifications",
  );

```

```

    await addDoc(notifCollectionRef, {
      id: `welcome-new-${Date.now()}`,
      read: false,
      text: "welcome",
      time: new Date().toISOString(),
      timestamp: new Date(),
      type: "info",
    });

    // Succès : Initialisation du compte local
    const userData = {
      uid: auth.currentUser.uid,
      name: cleanName,
      gender: gender,
      birthDate,
      age,
      img: getUniqueUserImage(auth.currentUser.uid, gender),
      gems: 100,
      isOnline: true,
      isVerified: true,
      country: serverData.country || "fr",
      createdAt: new Date().toISOString(),
    };

    setUser(userData);
    setStep(1);
    setActiveTab("home");

```

```

    showToast(t("welcome"));
  } else {
    // ERREUR SERVEUR
    alert(serverData.error || t("error_registration"));

    // Optionnel : Si banni (403), on déconnecte
    if (response.status === 403) auth.signOut();
  }
} catch (e) {
  console.error(e);
  showToast(t("error_server_connection"));
}
};

```

1.6 Achats (Système stripe)

La boutique de HeartMania dispose de 3 packs de Gems

Pack Découverte (100 Gems) : c'est un pack de gems gratuit (3 utilisations quotidiennes)

Pack Passion (500 Gems) : pack payant à 19.99€

Pack Élite (1000 Gems) : pack payant à 39.99€

```

const handleCheckout = async () => {
  if (isProcessing || cart.length === 0) return;
  setIsProcessing(true);

  try {
    const token = await auth.currentUser.getIdToken();
    const isFreePack = cart.some((item) => item.price === 0);

    // --- CAS 1 : C'EST GRATUIT ---
    if (isFreePack) {
      const response = await fetch(`${API_URL}/api/payment/add-gems`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${token}`,
        },
        body: JSON.stringify({
          amount: cart[0].gemAmount,
          locale: i18n.language,
          packName: cart[0].id,
          isDaily: true,
        }),
      });
    }
  }
};

```

```

if (response.ok) {
  const resData = await response.json();

  const userId = auth.currentUser.uid;

  // --- 1. AJOUT DE LA NOTIFICATION DANS LE PANNEAU LATÉRAL ---
  const notifCollectionRef = collection(
    db,
    "users",
    auth.currentUser.uid,
    "notifications",
  );

  await addDoc(notifCollectionRef, {
    id: `buy-free-${Date.now()}`,
    read: false,
    text: "checkout_success",
    pack: `pack_name_${cart[0].id}`,
    amount: cart[0].gemAmount,
    time: new Date().toISOString(),
    timestamp: new Date(),
    type: "shop",
  });

  // Si le serveur renvoie le nouveau solde de gemmes, on met à jour l'état local
  if (resData.gems !== undefined) {
    setGems(resData.gems);
  }
}

```

```

// On vide le panier et on retourne à l'accueil
setCart([]);
setActiveTab("home");

// --- 2. AFFICHAGE DE LA BULLE TOAST TRADUITE ---
showToast(
  t("notif_purchase_success", {
    pack: t(`pack_name_${cart[0].id}`),
    amount: cart[0].gemAmount,
  }),
);
} else {
  const errorData = await response.json();
  if (errorData.errorKey) {
    showToast(t(errorData.errorKey));
  } else {
    showToast(errorData.error || t("error_checkout"));
  }
}
}

```

```

//--- CAS 2: C'EST PAYANT (système Stripe)
else {
  // On ne stocke plus rien à l'avance pour éviter les faux positifs
  // si l'utilisateur annule.

  const response = await fetch(
    `${API_URL}/api/payment/create-checkout-session`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${token}`,
      },
      body: JSON.stringify({
        items: cart.map((item) => item.id),
        locale: i18n.language,
      }),
    },
  );
  const data = await response.json();
  if (data.url) {
    window.location.href = data.url;
  } else {
    throw new Error("URL de paiement non reçue");
  }
}
}

```

```

} catch (error) {
  console.error("Erreur checkout:", error);
  showToast(t("error_connection"));
} finally {
  setIsProcessing(false);
}
};

```

```

useEffect(() => {
  // 1. On récupère les paramètres directement depuis l'URL
  const searchParams = new URLSearchParams(window.location.search);
  const amountParam = searchParams.get("amount");
  const packParam = searchParams.get("pack");

  // Sécurité : Si on est sur une URL Stripe (/success ou /cancel) mais sans paramètres ou déjà nettoyée,
  // ou si l'utilisateur est en train de recharger, on s'assure de nettoyer la route.
  if (
    window.location.pathname.includes("success") ||
    window.location.pathname.includes("cancel")
  ) {
    if (amountParam && packParam) {
      try {
        const gemAmount = parseInt(amountParam, 10) || 0;

        let packLabel = packParam;
        if (packParam.toLowerCase().includes("passion"))
          packLabel = t("pack_name_passion");
        else if (packParam.toLowerCase().includes("elite"))
          packLabel = t("pack_name_elite");
        else if (
          packParam.toLowerCase().includes("decouverte") ||
          packParam.toLowerCase().includes("découverte")
        )
          packLabel = t("pack_name_discovery");
      }
    }
  }

```

```

    // 2. Affichage du toast
    showToast(
      t("notif_purchase_success", {
        pack: packLabel,
        amount: gemAmount,
      })
    );
  } catch (e) {
    console.error(
      "Erreur lors de l'affichage de la notification de succès:",
      e,
    );
  }
}

// 3. Nettoyage et redirection immédiate vers l'application principale
// window.history.replaceState nettoie visuellement l'URL en restant sur place
window.history.replaceState({}, document.title, "/");

// setActiveTab et navigate forcent React Router et ton layout à charger l'accueil
setActiveTab("home");
navigate("/");
}, [location.search, location.pathname, t, navigate]);

```

Ci-dessous l'aperçu quand on choisit un pack payant

Comme j'ai utilisé un environnement de test, il s'agit uniquement de données de simulations aucune transaction réelle n'est effectuée.

←

environnement de test Hear...

Environnement de test

Pack de Gemmes passion

5,00 €

Payer avec  link

OU

Coordonnées

E-mail

email@exemple.com

Moyen de paiement

 Carte

Informations de la carte

1234 1234 1234 1234

VISA    

MM / AA

CVC

 123

Nom du titulaire de la carte

Nom complet

Pays ou région

France

▼

Païement réussi !

Merci pour ton achat. Tes gemmes ont été ajoutées à ton compte.

[Retour à l'accueil](#)

Ci-dessous un aperçu des logs du Webhook Stripe:

30

sophisticated-inspiration Actif

<https://heartmanix.alwaysdata.net/api/payment/webhook>

Aperçu Événements envoyés

🔍 Trouver un événement à l'a	⊕ État
Aujourd'hui	
Mis à jour aujourd'hui 10:58:16 UTC Actualiser	
200 OK 🟢	checkout.session.completed 10:57:53
200 OK 🟢	checkout.session.completed 10:56:28
200 OK 🟢	checkout.session.completed 10:22:33

1.7 Gestion d'inactivité ,suppression des comptes et gestion des sessions

Je me suis bien assuré que si la même session est ouverte par un même utilisateur sur plusieurs appareils, il est automatiquement éjecté sur les autres onglets en exploitant la fonction qui surveille l'inactivité en permanence. Dès qu'il bouge sur la page l'écouteur vérifie s'il possède toujours un token valide. Comme ça on assure la sécurité et on réduit les tentatives de fraudes utilisateur.

```
// 1. 🛡 VERIFICATION SILENCIEUSE DU COMPTE (Anti-fantôme multi-appareil)
// Dès que le PC se réveille ou qu'on vérifie l'inactivité, on valide le token Auth
if (auth.currentUser) {
  try {
    // Force Firebase Auth à valider le compte en arrière-plan
    await auth.currentUser.getIdToken(true);
  } catch (error) {
    // Si le compte a été supprimé sur un autre appareil, l'erreur est captée ici !
    console.log("Le compte a été supprimé depuis un autre appareil. Nettoyage...");
    clearInterval(interval);
    await auth.signOut();
    window.location.href = "/"; // Éjection propre
    return;
  }
}
```

1.8. Suppression des comptes fantômes et ban

Les comptes qui ont eu un soft delete(suppression partielle : on vide leurs champs sans réelle suppression) sont supprimés automatiquement après les 48h passées via le fichier cron

J'ai mis en place un système de bannissement si un user a son attribut `isBanned` en `true` il est banni définitivement et il pourra plus se reconnecter.

2. La conception du backend

J'ai installé la dépendance `geoip-lite` qui m'a permis de récupérer les pays des utilisateurs en se référant à leurs adresses ip respectives. Contrairement à des API de géolocalisation tierces qui nécessitent une requête réseau externe à chaque appel, `geoip-lite` interroge une base de données locale, garantissant ainsi une latence minimale.

J'ai structuré mon code de backend en MVC (Model, View, Controller) avec en plus un fichier de configuration `ecosystem.config.js` qui me permet de lancer deux choses en parallèle (le code server et le cron job)

Ci-dessous un tableau qui illustre les endpoints de l'API utilisé en backend

Contrôleur	Endpoint	Méthodes HTTP	Description
UserController	/profiles	GET	Récupère la liste des profils
UserController	/:uid	GET	Récupère les détails d'un utilisateur spécifique via son UID.
UserController	/update-status	POST	Met à jour le statut en ligne/hors ligne ou le statut d'activité de l'utilisateur.
UserController	/delete-account	DELETE	Supprime définitivement les données utilisateur, les collections liées et le compte Auth.
UserController	/update-language	POST	Change la langue de l'interface utilisateur en base de données.
UserController	/update-bio	POST	Met à jour le texte de la biographie d'un profil
UserController	/toggle-favorite	POST	Ajoute ou retire un utilisateur de la liste des favoris (Match).
UserController	/super-like	POST	Envoyer un superlike à un profil
chatController	/send-message	POST	Envoie un message
chatController	/mark-read	POST	Marquer un message comme lu
chatController	/report-user	POST	Signale un profil utilisateur inapproprié
authController	/register	POST	Crée un nouveau compte utilisateur

paymentController	/webhook	POST	Reçoit et traite les notifications de paiement Stripe
paymentController	/create-checkout-session	POST	Initialise une session de paiement Stripe pour l'achat de gemmes
paymentController	/add-gems	POST	Ajoute les gems au solde de l'utilisateur après un paiement Stripe

Configuration

L'application HeartMania utilise une base de données cloud (NoSQL avec Firestore)

Ci-dessous le fichier de configuration qui permet de communiquer avec la base de données Firebase.

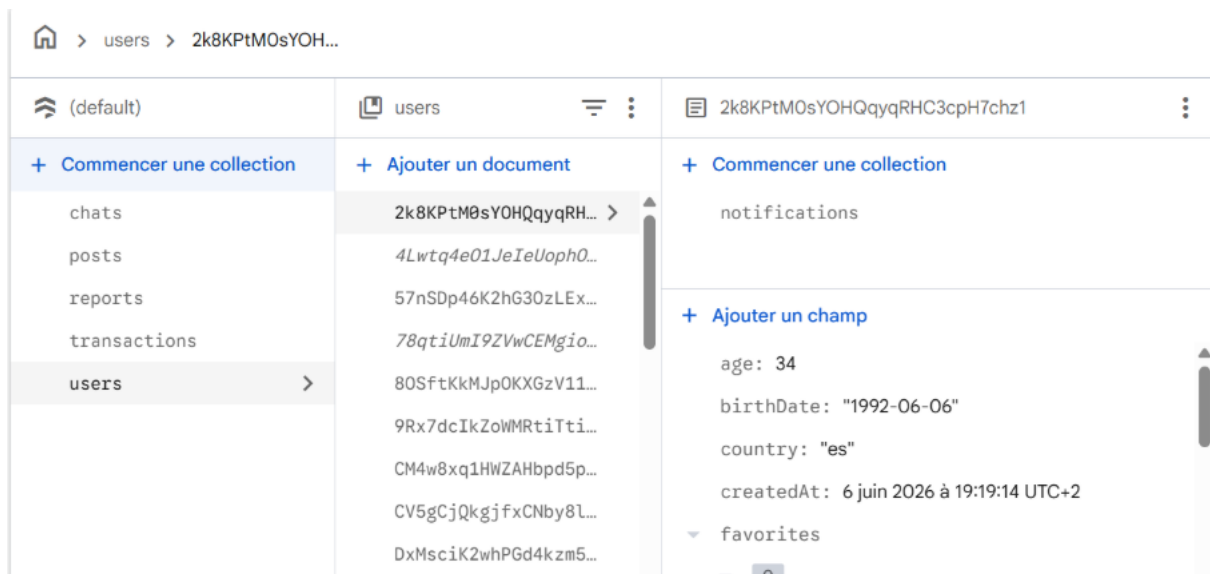
```

1  import { initializeApp } from "firebase/app";
2  import { getAuth, GoogleAuthProvider } from "firebase/auth";
3  import { getFirestore } from "firebase/firestore";
4  // On utilise les variables d'environnement Vite pour ne pas exposer les clés
5  const firebaseConfig = {
6    apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
7    authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
8    projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
9    storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
10    messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
11    appId: import.meta.env.VITE_FIREBASE_APP_ID,
12    measurementId: import.meta.env.VITE_FIREBASE_MEASUREMENT_ID,
13  };
14
15  // Initialisation
16  const app = initializeApp(firebaseConfig);
17
18  // Export des services dont le front a besoin
19  export const auth = getAuth(app);
20  export const googleProvider = new GoogleAuthProvider();
21  googleProvider.setCustomParameters({
22    prompt: 'select_account'
23  });
24
25  export const db = getFirestore(app);

```

Structure de la base de données Firestore

La base de données est organisée en collections de documents. Chaque collection centralise un type d'entité spécifique pour garantir la cohérence des données et optimiser les requêtes.



1. Collection users

C'est le cœur de l'application. Chaque document utilisateur est identifié par l'uid unique généré par Firebase Auth.

• Champs :

- uid (String) : Identifiant unique de l'utilisateur.
- name (String) : Nom ou pseudonyme.
- gender (String) : Genre.
- birthDate (String) : Date de naissance.
- age (Number) : Age.
- isAnonymous (Boolean) : Indicateur s'il s'agit d'un compte invité.
- img (String) : URL de la photo de profil.
- gems (Number) : Solde actuel de gemmes.
- IsDeleted(Boolean):Statut de suppression.
- isOnline (Boolean) : État de connexion en temps réel.
- isBanned (Boolean) : Statut de bannissement.
- country (String) : Code pays (détecté via geoip-lite).
- isVerified (Boolean) : Statut de vérification du compte.
- dailyClaims (Map) : Suivi des bonus quotidiens
- createdAt (Timestamp) : Date d'inscription.
- lastSeen (Timestamp) : Date de dernière activité.
- favorites (Array) : Liste des IDs des profils favoris.
- profileViews (Array) : Historique ou compteurs des visites sur le profil.
- notifHistory (Array) : Historique des notifications reçues.
- hiddenChats (Array) : Liste des IDs des conversations masquées.

Sous-collection notifications :

Contient les documents de notifications individuelles (achat, superlike, etc.).

2. Collection transactions

Cette collection est cruciale pour l'intégrité financière et l'idempotence des paiements.

- **Identifiant du document** : Correspond au session_id unique généré par Stripe.
- **Champs** :
 - uid (String) : Identifiant de l'utilisateur ayant effectué l'achat.
 - processedAt (Timestamp) : Date de validation du paiement par le Webhook.

3. Collection chats

Cette collection gère les échanges privés entre les utilisateurs. Elle repose sur une structure simple où chaque document représente un message ou une conversation persistante identifiée par un combinedId.

- **Identifiant du document** : Le combinedId (concaténation triée des uid des deux participants, ex: "uidA_uidB"). Ce mécanisme garantit qu'une seule instance de conversation existe entre deux utilisateurs donnés.
- **Champs** :
 - combinedId (String) : Identifiant unique de la conversation.
 - createdAt (Timestamp) : Date et heure de création/envoi du message.
 - senderId (String) : L'uid de l'utilisateur ayant envoyé le message.
 - status (String) : État du message (ex: "read").
 - text (String) : Contenu textuel du message.

4. Collection reports

Cette collection est dédiée à la modération de la plateforme. Elle permet de centraliser les signalements effectués par les utilisateurs pour permettre une intervention ultérieure des administrateurs.

- **Champs** :
 - reporterUserId (String) : uid de l'utilisateur qui effectue le signalement.
 - reportedUserId (String) : uid de l'utilisateur visé par le signalement.
 - reason (String) : Motif court du signalement.
 - description (String) : Détails complémentaires fournis par l'utilisateur.
 - timestamp (Timestamp) : Date et heure du signalement.
 - status (String) : État du traitement ("pending", "reviewed", "resolved").

5. Collection posts

Cette collection gère le fil d'actualité. Elle utilise la dénormalisation pour permettre un affichage rapide des contenus et des interactions.

- **Champs :**

- `userId (String)` : uid de l'auteur.
- `content (String)` : Contenu de la publication.
- `createdAt (Timestamp)` : Date de création.
- `authorName (String)` : Nom de l'auteur.
- `authorImg (String | null)` : URL de l'image de profil.
- `comments (Array)` : Tableau stockant tous les commentaires associés au post.

Sécurité et Contrôle d'Accès : Règles Firestore

Pour garantir l'intégrité des données et la confidentialité des échanges, l'accès à la base de données Firestore est régi par des *Règles de Sécurité Firestore*. Ces règles filtrent chaque requête directement sur les serveurs de Firebase, interdisant toute lecture ou écriture non autorisée, indépendamment du code front-end.

L'architecture de sécurité repose sur un principe fondamental : Le front-end n'a des droits d'écriture que sur ce qu'il possède ou contrôle directement. Les données sensibles sont verrouillées et réservées exclusivement au backend (Admin SDK).

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     match /reports/{reportId} {
5       // Autorise la création de signalements uniquement si l'utilisateur est authentifié
6       allow create: if request.auth != null;
7       // Optionnel : Seuls les admins peuvent lire les rapports
8       allow read: if request.auth != null && get(/databases/{database}/documents/reports/{reportId}).data.role == 'admin';
9     }
10    match /posts/{postId} {
11      allow read: if request.auth != null;
12
13      // Seuls les vrais comptes (pas anonymes) peuvent créer un post
14      allow create: if request.auth != null && request.auth.token != null;
15
16      // Sécurisation stricte des modifications (Likes / Commentaires)
17      allow update: if request.auth != null && (
18        // Règle 1 : Seul l'auteur d'un post peut le modifier
```

❖ Collection reports (Signalements)

Cette règle structure le flux de modération en protégeant l'anonymat des signalements.

- **Création (create)** : Autorisée à tout utilisateur authentifié (`request.auth != null`).
- **Lecture (read)** : Entièrement verrouillée pour les utilisateurs standards. Seuls les comptes disposant du drapeau de privilège `isAdmin == true` dans leur propre document `users` peuvent consulter les rapports (utilisation de la méthode dynamique `get()`).

❖ Collection posts (Fil d'actualité)

La logique de cette collection sépare le droit de consultation du droit d'interaction sociale.

- **Lecture (read)** : Accessible à tout utilisateur connecté.
- **Création (create)** : Réservée exclusivement aux utilisateurs s'étant connectés via un fournisseur officiel (Google, Téléphone, Email). Les comptes anonymes sont explicitement bloqués (`sign_in_provider != 'anonymous'`).
- **Suppression (delete)** : Protection stricte ; seul l'auteur du post (`resource.data.userId`) peut le supprimer.
- **Mise à jour (update)** : Gère deux cas de figure distincts :
 - L'auteur d'origine a tous les droits de modification sur son post.
 - Un tiers (Visiteur) peut interagir (liker ou commenter) à condition que les champs fondamentaux (`content` et `userId`) restent strictement inchangés. De plus, un utilisateur anonyme se voit interdire l'ajout ou la modification de commentaires (`comments`).

❖ Collection users et sous-collection notifications

A. Profils Utilisateurs (Document Racine)

- **Lecture (read)** : Publique pour tous les utilisateurs connectés (nécessaire pour la recherche et l'affichage des profils).
- **Création et Suppression (create, delete)** : Strictement restreintes à l'utilisateur dont l'UID correspond au nom du document (`request.auth.uid == userId`).
- **Mise à jour (update)** : Divisée en deux scénarios pour éviter les failles d'élévation de privilèges :
 - Gestion de son propre profil : L'utilisateur peut modifier ses informations de base (`name`, `img`, `gender`, etc.) mais le système applique une politique de contrôle des clés (`diff().affectedKeys()`). Il est interdit d'altérer arbitrairement le solde de gemmes (`gems`) sauf si l'opération maintient un solde positif.
 - Interaction par un tiers : Un utilisateur externe n'a le droit de modifier sur un profil ciblé **uniquement** les tableaux historiques de traçabilité : `notifHistory` (historique), `favorites` (mise en favoris) et `profileViews` (compteur de vues).

B. Sous-collection Imbriquée notifications

L'arborescence utilise le chemin `/users/{userId}/notifications/{notifId}` :

- **Création (create) :** Tout utilisateur connecté peut pousser une notification (ex: avertir un utilisateur qu'il a reçu un message ou un intérêt).
- **Lecture, Mise à jour et Suppression (read, update, delete) :** Exclusivement réservées au propriétaire du compte parent (`request.auth.uid === userId`). L'accès en mise à jour (`update`) est spécifiquement ouvert pour permettre au front-end d'exécuter des opérations de traitement par lot (*Batched Writes*) afin de faire passer le statut d'une notification à `read: true`.

❖ Collection chats (Messagerie Privée)

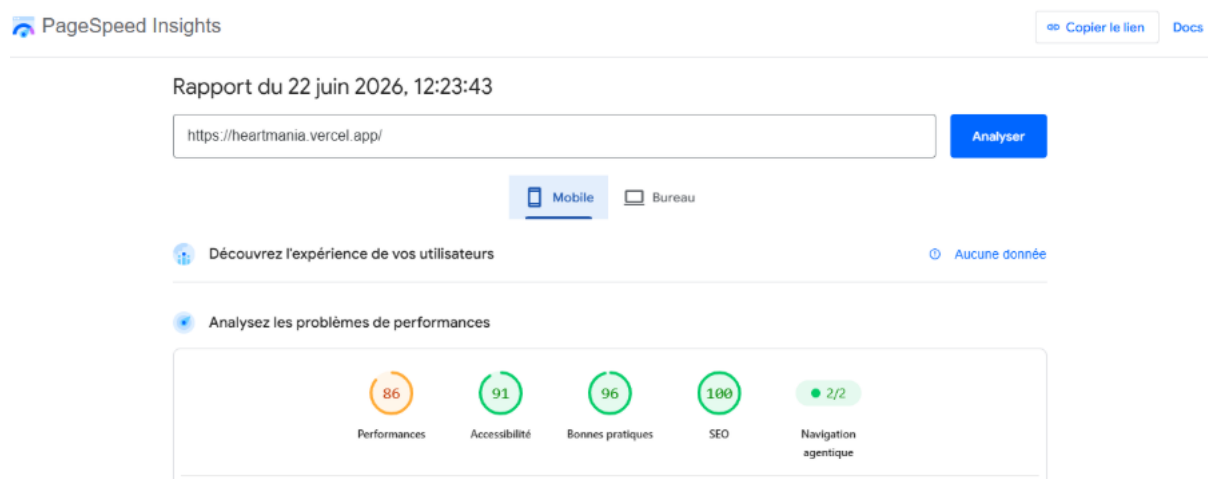
La sécurité de la messagerie garantit la confidentialité de bout en bout et empêche l'injection de données corrompues.

- **Création (create) :** Un message ne peut être créé que si l'expéditeur déclaré est bien l'utilisateur authentifié (`senderId === request.auth.uid`). Le texte doit obligatoirement être de type string et limité à un maximum de 1000 caractères pour éviter la saturation de la base de données.
- **Lecture (read) :** Un utilisateur ne peut lire un message que s'il est l'expéditeur direct (`senderId`) ou s'il fait techniquement partie de la conversation en extrayant et vérifiant sa présence dans l'identifiant unique de la discussion (`request.auth.uid` in `resource.data.combinedId.split('_')`).
- **Mise à jour (update) :** Restreinte aux deux membres du chat. L'opérateur `.hasOnly(['status'])` verrouille intégralement le document : seul le champ de statut de lecture (`status`) peut être modifié (pour marquer un message comme "read"). Le contenu textuel ou l'expéditeur original deviennent immuables.
- **Suppression (delete) :** Désactivée à la source (`if false`), assurant la persistance et l'historique permanent des échanges.

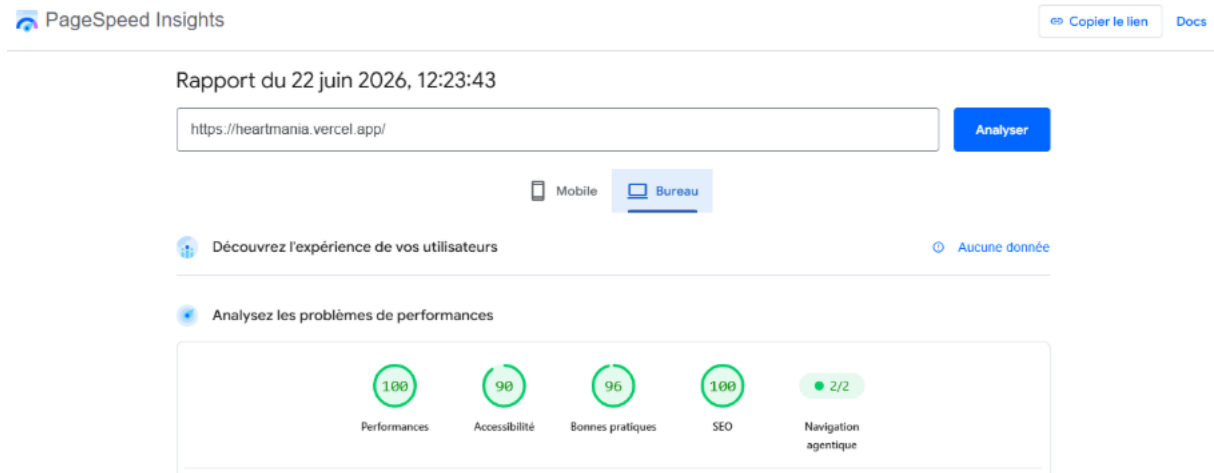
3. Performance & accessibilité

On retrouve ci-dessous les scores Lighthouse

Pour mobile :



Pour desktop :



Ces excellents scores Lighthouse s'expliquent par une stratégie d'optimisation poussée au niveau du Frontend. Le score parfait de 100/100 sur Bureau et la réactivité sur Mobile (86/100) découlent directement de l'intégration du lazy loading, permettant de ne charger les composants et les images que lorsqu'ils apparaissent à l'écran, ainsi que de la conversion des assets visuels au format webp pour réduire drastiquement le poids des fichiers, le tout propulsé par un build de production ultra-optimisé et léger grâce à Vite. Enfin, l'obtention des notes maximales en SEO (100) et en Accessibilité (91) est le résultat d'une structure sémantique HTML5 stricte associée au choix rigoureux des contrastes de couleurs via la palette numérique de TailwindCSS et à l'intégration systématique d'attributs alt pertinents sur toutes les images, garantissant une indexation parfaite par les moteurs de recherche et une compatibilité optimale avec les lecteurs d'écran pour les utilisateurs malvoyants.

Conclusion

Ce travail a été le résultat de mois de développement et de recherche. J'ai rencontré plusieurs difficultés tout au long du développement de ce projet, que ce soit au niveau de codage ou du débogage ou même du choix de l'hébergement.

J'ai mené seul ce projet, du concept initial à la mise en production, apprenant à structurer, à corriger, à affiner, jusqu'à trouver le bon résultat. Au fil du développement, j'ai découvert ce que signifiait vraiment construire une application vivante : penser pour l'utilisateur, anticiper ses gestes, traduire une idée abstraite en expérience concrète.

J'ai énormément appris pendant la réalisation de ce travail. J'ai pu mettre en pratique mes connaissances et les améliorer.